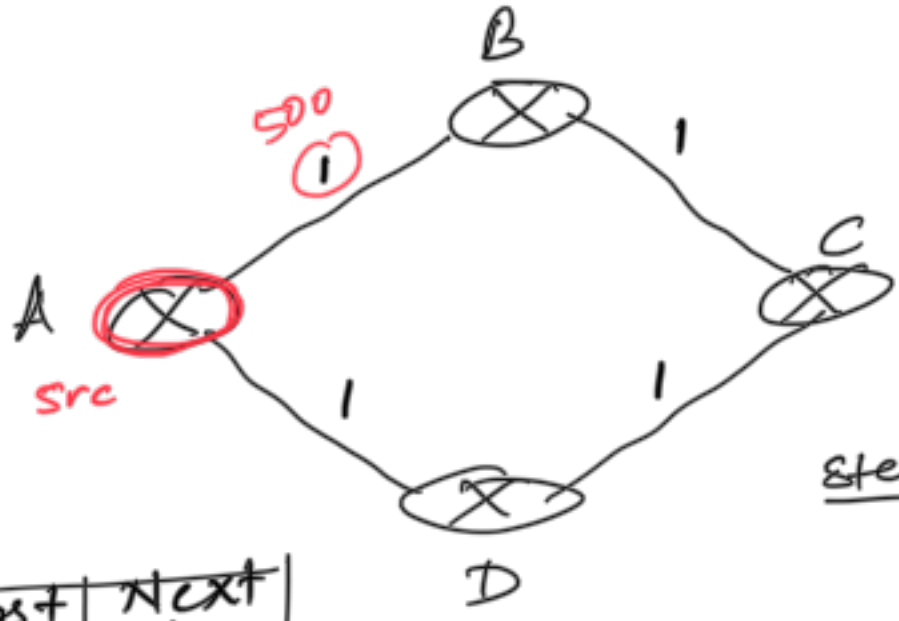
Lec 24: Dynamic Routing Algorithms

13/11/23

* Link state Routing (LSR) overview



RT: A

Network	cost	Next hop
A	0	—
B	1	B
C	∞	—
D	1	D

Step1: RT initialization
a) self cost = 0
b) add direct link cost

Step 2: RT is broadcasted (shared with entire n/w)
global view
- sharing is done
  - change
  - periodic ??

Step 3: Received RTs from entire n/w
- Compute RT using Dijkstra's algo.

Problem: Route oscillations [Refer Slides]
Solution ↗

Dynamic Routing

Distance Vector Routing

- Each router exchanges its distance vector (subset of RT) only with the neighbors
  - updates are sent
    - → change
    - → periodic
- Any changes in link cost travel one hop at a time, convergence is slow.
- Routing loops ⇒ Count-to-infinity problem

Solutions:
1) Split horizon
2) Poisoned reverse

- Protocol that implement DVR
  - ↳ RIP uses hop count as the link cost metric

max hop count = 16 = ∞

Link State Routing

- Each router obtains information about directly connected links/nodes (a.k.a. neighbors) & exchanges the neighbor info with entire network (broadcast)
- Since link state updates are broadcasted, convergence is fast
- Route oscillations are possible
- Protocol that implements LSR
  - ↳ OSPF

---

* Exterior routing ⇒ BGP

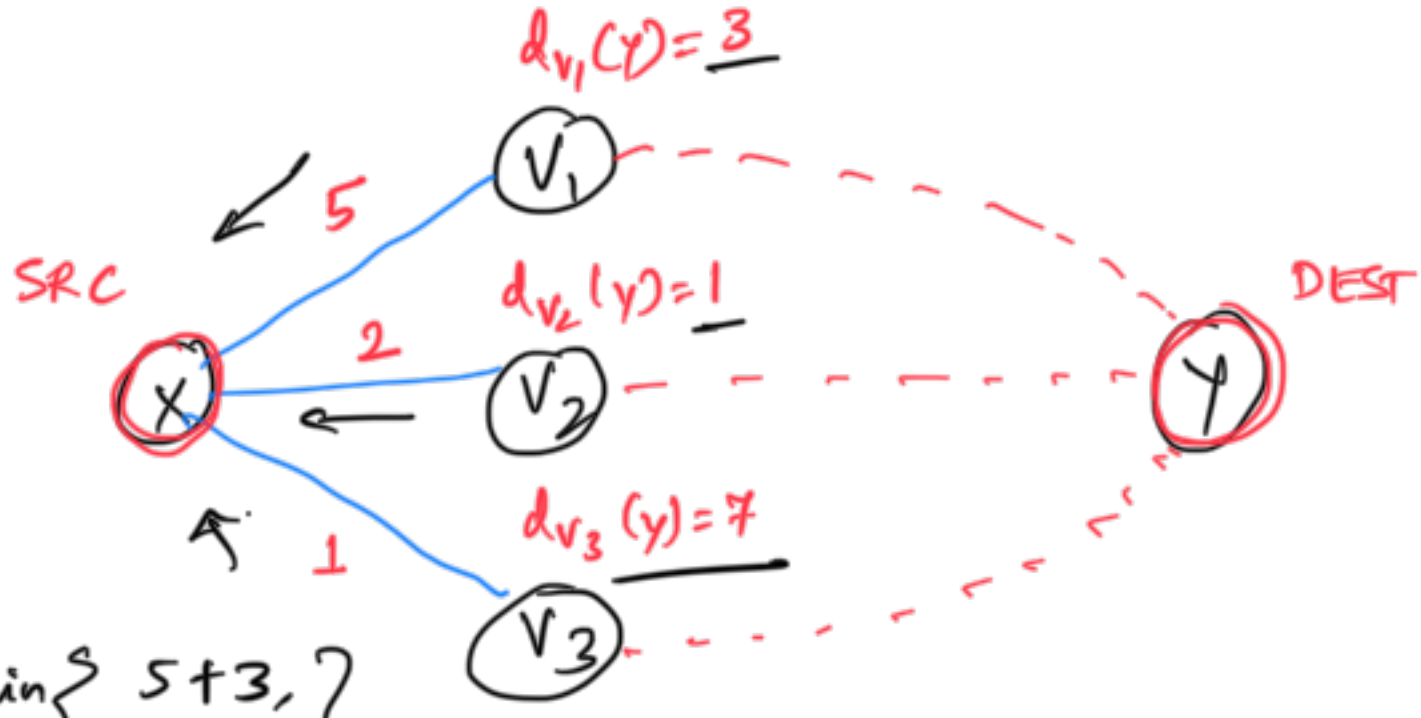
* Distance Vector Routing algorithm

- iterative — iterates until routes converge; no external trigger needed to stop the algorithm
- asynchronous — not necessary that all routers share their DVs at the same time
- distributed — Each router receives DV from neighbors, computes its own DV, shares the computed changes with the neighbors

* DV algorithm: based on Bellman-ford equation

$$d_x(y) = \min_{v} \{ c(x,v) + d_v(y) \}$$

- $d_x(y)$: least cost path from node x to node y
- $\min_v$: set of neighbors of node 'x'
- $c(x,v)$: cost to reach the neighbor 'v' from 'x'
- $d_v(y)$: least cost path from 'v' to 'y'



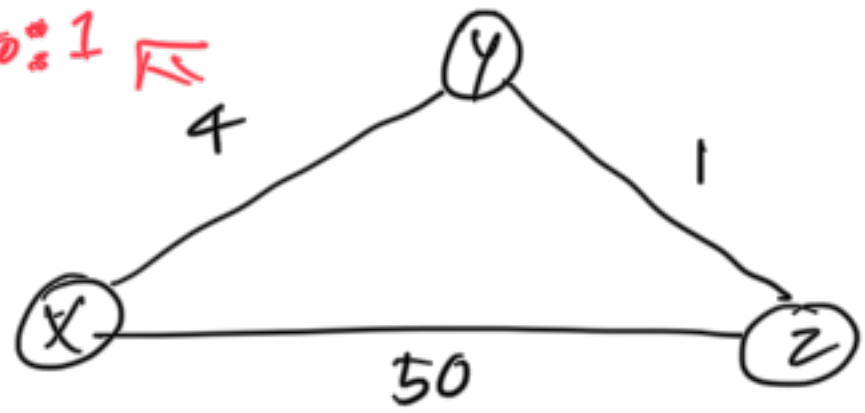
$d_x(y) = \min \{ 5+3, 2+1, 1+7 \}$

$d_x(y) = 3$

* DVR algorithm : Link cost changes & Link failure

	x	y	z
x			
y	4	0	1
z			

to: 1



	x	y	z
x	0	4	5
y			
z			

	x	y	z
x			
y			
z	5	1	0

Count-to-infinity problem

Good news travel fast ⇒ link cost decreases

Bad news take longer to spread ⇒ link cost increases

* X to Y cost changes from 4 to 1 Good news

- Started with converged routes
- Took 2 iterations after link cost change to converge

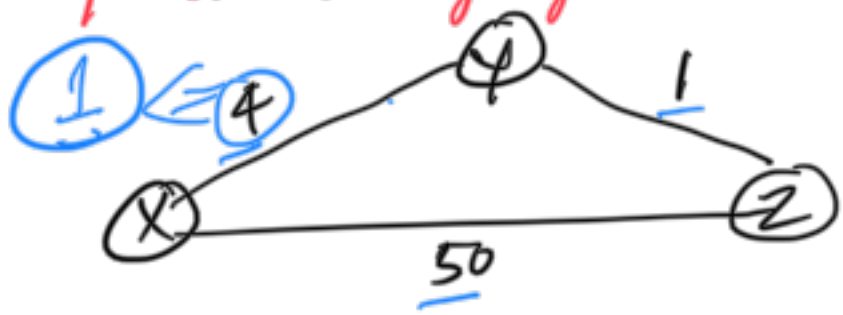


Table for X

X	X	Y	Z
X	0	~~4~~ 1	~~5~~ 2
Y	~~4~~ 1	~~0~~ 0	~~1~~ 1
Z	5	1	0

X	X	Y	Z
X	0	1	2
Y	1	0	1
Z	2	1	0

X	X	Y	Z
X			
Y			
Z			

Table for Y

Y	X	Y	Z
X	0	1	5
Y	~~4~~ 1	0	1
Z	~~5~~ 6	1	0

Y	X	Y	Z
X	0	1	2
Y	1	0	1
Z	2	1	0

Y	X	Y	Z
X			
Y			
Z			

Table for Z

Z	X	Y	Z
X	0	1	5
Y	1	0	1
Z	~~5~~ 2	1	0

Z	X	Y	Z
X	0	1	2
Y	1	0	1
Z	2	1	0

Z	X	Y	Z
X			
Y			
Z			

∴ No change from prev iteration, routes have converged

* x to y cost changes from 4 ~~to 60~~

44 iterations to converge

- started with converged routes
- will take 44 iterations to converge after link cost increased to 60

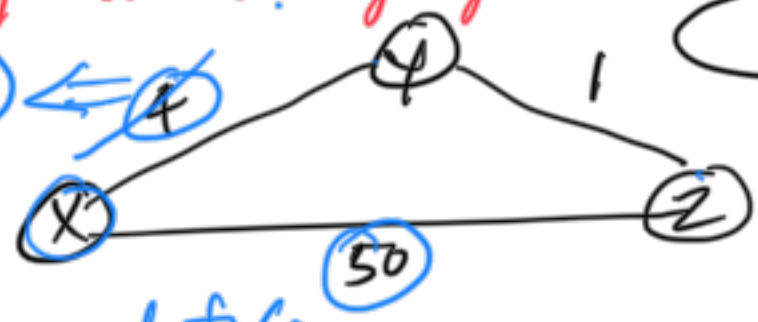


Table for X

X	X	Y	Z
X	0	~~4~~ 60	5
Y	4	0	1
Z	5	1	0

X	X	Y	Z
X	0	51	5
Y	60	0	1
Z	5	1	0

X	X	Y	Z
X	0	60	50
Y	6	0	1
Z	50	1	0

Table for Y

X	X	Y	Z
X	0	4	5
Y	~~4~~ 60	0	1
Z	5	1	0

X	X	Y	Z
X	0	60	5
Y	6	0	1
Z	5	1	0

X	X	Y	Z
X	0	51	5
Y	51	0	1
Z	50	1	0

Table for Z

X	X	Y	Z
X	0	4	5
Y	4	0	1
Z	5	1	0

X	X	Y	Z
X	0	60	5
Y	60	0	1
Z	50	1	0

X	X	Y	Z
X	0	51	5
Y	6	0	1
Z	7	1	0

* Solution to Count-to-infinity problem

Root cause: A router 'R' considers the update from a neighbors whose routes were decided using 'R's update. This causes routing loops

Solution:

If a router 'X' encorporates DV update from neighbor 'v' for an entry to dest 'd' then 'X' should never send an update for dest 'd' to router 'v' DV

a) Split horizon

b) Poisoned reverse

same as (a) but DV update is sent with cost